Related Data Loading in EF Core

Eager Loading • Lazy Loading • Explicit Loading

Core idea: These three strategies determine **when and how** EF Core loads related entities through navigation properties.

1. Eager Loading

Definition: Load related data as part of the initial query using **Include()**.

```
var students = context.Students
    .Include(s => s.Enrollments)
    .ToList();
```

Behavior: EF Core retrieves Students and Enrollments together. SQL often uses a JOIN for this collection navigation (the exact SQL depends on the query/provider).

Advantages: predictable data access; fewer hidden queries; easier to inspect and optimize.

Disadvantages: may fetch data you do not need; large joins or multiple included collections can produce heavy results.

2. Lazy Loading

Definition: Related data is loaded automatically when its navigation property is accessed, if it has not already been loaded.

```
var students = context.Students.ToList();

foreach (var student in students)
{
    Console.WriteLine(student.Enrollments.Count);
}
```

Behavior: The first query loads Students. Accessing each student's Enrollments can trigger another query.

N+1 example: 1 query for Students + 1 query per student. For 100 students, that can mean 101 queries.

Advantages: convenient coding experience; can be acceptable for small apps or small datasets.

Disadvantages: hidden SQL; less predictable performance; risky inside loops.

3. Explicit Loading

Definition: You decide exactly when to load related data by calling **Load()** manually.

```
var student = context.Students
    .First(s => s.StudentId == 1);

context.Entry(student)
    .Collection(s => s.Enrollments)
    .Load();
```

For a reference navigation property:

```
context.Entry(enrollment)
    .Reference(e => e.Student)
    .Load();
```

Behavior: Load the main entity first, then explicitly request its related collection or reference when needed.

Advantages: precise control over timing; predictable query execution; useful for conditional loading; avoids hidden lazy-loading queries.

Disadvantages: more manual code; easy to forget the Load() call.

4. Side-by-Side Comparison

Strategy	When data loads	Main trade-off
Eager	With the initial query	Predictable, but can over-fetch
Lazy	When navigation is accessed	Convenient, but can cause N+1

Explicit	When you call Load()	Controlled, but requires extra code

5. Real TrainingCenterDB Scenarios

Student Details Page Student + Enrollments + Courses	**Eager Loading** The page needs the related data together.
Small Internal Tool Related data is accessed occasionally	**Lazy Loading** May be acceptable for small, controlled workloads.
Show Courses button Load Enrollments only after a click	**Explicit Loading** Load only when the user requests it.

6. Common Mistakes

• Using Lazy Loading inside loops without checking the generated SQL → possible N+1.

• Using Include() everywhere → unnecessary data and potentially heavy queries.

• Forgetting Load() with Explicit Loading → the navigation may remain unloaded.

• Assuming navigation-property access is always in-memory → with Lazy Loading, it may execute SQL.

Professional Guidance

Prefer predictable data access: use Eager Loading when the required related data is known, Projection when you only need selected fields, and Explicit Loading when loading should happen conditionally. Use Lazy Loading carefully and monitor its SQL.

Remember: Eager = load now • Lazy = load on access • Explicit = load when you command it.